# Effective Dataset Distillation for Spatio-Temporal Forecasting with Bi-dimensional Compression (Supplementary Material)

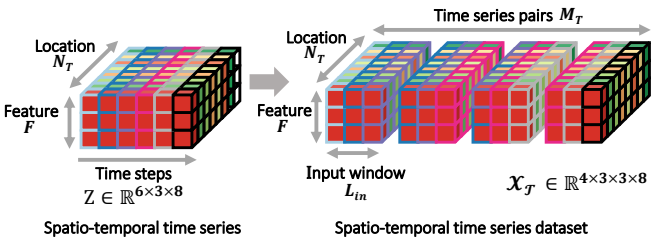



Fig. 8. An example of a spatio-temporal time series dataset constructed from spatio-temporal time series. Entries with the same color correspond to data from the same location, and entries with the same boundary color correspond to data from the same time step. Refer to Example 3 for details.

### A. Illustrative Example of Dataset Construction

A toy example showing how to construct a spatio-temporal time-series dataset $\mathcal{X}_\mathcal{T}$ from a spatio-temporal time series $Z$ is provided below.

**Example 3.** *In Figure 8, we illustrate how to build $\mathcal{X}_\mathcal{T}$ from $Z$. In this example, $Z(0:3)$ becomes $X_0$, $Z(1:4)$ becomes $X_2$, $Z(2:5)$ becomes $X_3$, and $Z(3:6)$ becomes $X_4$. Then, $\mathcal{X}_\mathcal{T}$ is set to $(X_t)_{t=0}^3$.*

### B. Complexity Analysis

This section provides detailed proofs of Theorems and Lemmas, as well as the complexity analysis of location clustering.

**Model training on synthetic datasets:** Our analysis builds upon Lemma 1, which describes the complexity of MTGNN.

**Lemma 1** (Time Complexity of MTGNN)**.** *Given location embeddings from the location encoder, a forward and backward pass of MTGNN on the synthetic dataset $\mathcal{S}$ takes $O(N_\mathcal{S}^2 + N_\mathcal{S} L_{in})$ time.*

*Proof.* The proof for the forward pass is based on the derivation provided in Appendix A.1 of [1], and the backward pass has the same complexity up to a constant factor. □

**Theorem 1** (Time Complexity of Model Training)**.** *Algorithm 1 takes $O(T_{model} M_\mathcal{S}(\frac{N_\mathcal{S}^2}{K} + N_\mathcal{S} L_{in}))$ time.*

*Proof.* The time complexity is determined by two components: our location encoder and the MTGNN model. The most dominant part in the location encoder is the self-attention layer, which takes $O(\frac{N_\mathcal{S}^2}{K^2})$ time. By Lemma 1, the time complexity of the MTGNN backbone is $O(\frac{N_\mathcal{S}^2}{K^2} + \frac{N_\mathcal{S}}{K} L_{in})$. Thus, training on a single location set takes $O(M_\mathcal{S}(\frac{N_\mathcal{S}^2}{K^2} + \frac{N_\mathcal{S}}{K} L_{in}))$ time, and training on all location sets takes $O(M_\mathcal{S}(\frac{N_\mathcal{S}^2}{K} + N_\mathcal{S} L_{in}))$ time. Finally, training for $T_{model}$ epochs takes $O(T_{model} M_\mathcal{S}(\frac{N_\mathcal{S}^2}{K} + N_\mathcal{S} L_{in}))$ time. □

**Theorem 2** (Space Complexity of Model Training)**.** *Algorithm 2 requires $O(N_\mathcal{S} F(L_{in} + L_{out}) M_\mathcal{S} + \frac{N_\mathcal{S}^2}{K^2})$ space.*

*Proof.* Saving the synthetic dataset requires $O(N_\mathcal{S} F(L_{in} + L_{out}) M_\mathcal{S} + \frac{N_\mathcal{S}^2}{K^2})$ space. The location encoders require $O(\frac{N_\mathcal{S}^2}{K^2})$ space during the computation of the self-attention layer. There are three main components in MTGNN: the graph learning layer, the graph convolution module, and the temporal convolution module (refer to [1] for their details). The graph learning layer requires $O(\frac{N_\mathcal{S}^2}{K^2})$ space when building an adjacency matrix. Storing the activation values in the graph convolution module requires $O(M_\mathcal{S}(\frac{N_\mathcal{S}}{K}))$ space. Storing the activation values in the temporal convolution layer requires $O(M_\mathcal{S}(\frac{N_\mathcal{S}}{K}) L_{in})$ space. In summary, the total space complexity becomes $O((\frac{N_\mathcal{S}}{K}) F L_{in} M_\mathcal{S} + \frac{N_\mathcal{S}^2}{K^2})$. □

**Location clustering:** Next, we analyze the complexity of location clustering, a key component of STemDist, in Lemma 2, where $T_{kmeans}$ denotes the number of K-means iterations.

**Lemma 2** (Time Complexity of Clustering Complexity)**.** *Algorithm 3 takes $O(N_\mathcal{T} N_\mathcal{S}\ (M_\mathcal{T} L_{in} F) T_{kmeans})$ time.*

*Proof.* The most dominant part of Algorithm 3 is K-means clustering (line 2). Within the K-means procedure, the most dominant step is computing the distances between the feature vectors of all pairs of clusters and locations in the original dataset, which is repeated in every iteration. Since the feature dimension is $M_\mathcal{T} L_{in} F$ and the number of pairs is $N_\mathcal{T} N_\mathcal{S}$, it takes $O(N_\mathcal{T} N_\mathcal{S} (M_\mathcal{T} L_{in} F) T_{kmeans})$ time. □

Regarding space complexity, $O((N_\mathcal{T} + N_\mathcal{S}) M_\mathcal{T} L_{in} F)$ space is required to store the $M_\mathcal{T} L_{in} F$-dimensional feature vectors of $N_\mathcal{T}$ locations and $N_\mathcal{S}$ clusters.

**Dataset distillation by STemDist:** We analyze the overall data-distillation cost of STemDist in Theorem 3.

**Theorem 3** (Time Complexity of STemDist)**.** *The time complexity of Algorithm 1, the whole distillation process, is $O(N_\mathcal{T} N_\mathcal{S} (M_\mathcal{T} L_{in} F) T_{kmeans} + T_{outer} T_{distill}((T_{model} M_\mathcal{S} + B)(\frac{N_\mathcal{S}^2}{K}) + (L_{in}(T_{model} M_\mathcal{S} + B) + (M_\mathcal{S} + B) L_{out} F_{out}) N_\mathcal{S}))$.*

*Proof.* Algorithm 1 begins with clustering (line 1), whose time complexity, $O(N_\mathcal{T} N_\mathcal{S} (M_\mathcal{T} L_{in} F) T_{kmeans})$, is derived

TABLE VI

CROSS-MODEL GENERALIZATION OF DATASET-DISTILLATION METHODS. THE SYNTHETIC DATASET PRODUCED BY EACH METHOD IS USED TO TRAIN VARIOUS MACHINE LEARNING MODELS FOR EVALUATION. THE PERFORMANCES OF OUR METHODS ARE HIGHLIGHTED IN BOLD; AND THE BEST PERFORMANCES IN EACH SETTING ARE HIGHLIGHTED IN YELLOW. O.O.M.: OUT OF MEMORY.

**AGCRN [2]**

| Method | GBA | | GLA | | ERA5 | | CA | | CAMS | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Relative MAE | Relative RMSE | Relative MAE | Relative RMSE | Relative MAE | Relative RMSE | Relative MAE | Relative RMSE | Relative MAE | Relative RMSE |
| K-Center [3] | 0.339 ± 0.035 | 0.417 ± 0.040 | 0.282 ± 0.016 | 0.353 ± 0.015 | O.O.M | O.O.M | O.O.M | O.O.M | O.O.M | O.O.M |
| DC [4] | 0.398 ± 0.094 | 0.476 ± 0.101 | 0.323 ± 0.058 | 0.389 ± 0.057 | O.O.M | O.O.M | O.O.M | O.O.M | O.O.M | O.O.M |
| CondTSF [5] | 0.392 ± 0.070 | 0.456 ± 0.066 | 0.313 ± 0.034 | 0.417 ± 0.055 | O.O.M | O.O.M | O.O.M | O.O.M | O.O.M | O.O.M |
| **STemDist** | **0.225 ± 0.004** | **0.286 ± 0.004** | **0.197 ± 0.003** | **0.261 ± 0.006** | **0.368 ± 0.003** | **0.412 ± 0.003** | **0.206 ± 0.006** | **0.267 ± 0.005** | **0.508 ± 0.005** | **0.669 ± 0.010** |
| Original data | 0.150 ± 0.002 | 0.197 ± 0.008 | 0.144 ± 0.005 | 0.187 ± 0.002 | O.O.M | O.O.M | O.O.M | O.O.M | O.O.M | O.O.M |

**STNORM [6]**

| Method | GBA | | GLA | | ERA5 | | CA | | CAMS | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Relative MAE | Relative RMSE | Relative MAE | Relative RMSE | Relative MAE | Relative RMSE | Relative MAE | Relative RMSE | Relative MAE | Relative RMSE |
| K-Center [3] | 0.359 ± 0.026 | 0.437 ± 0.024 | 0.281 ± 0.028 | 0.365 ± 0.023 | 0.496 ± 0.021 | 0.532 ± 0.019 | 0.304 ± 0.040 | 0.386 ± 0.040 | 0.820 ± 0.086 | 0.942 ± 0.066 |
| DC [4] | 0.357 ± 0.050 | 0.424 ± 0.039 | 0.305 ± 0.036 | 0.382 ± 0.040 | 0.410 ± 0.010 | 0.452 ± 0.007 | 0.306 ± 0.035 | 0.377 ± 0.030 | 0.589 ± 0.013 | 0.709 ± 0.014 |
| CondTSF [5] | 0.399 ± 0.091 | 0.465 ± 0.077 | 0.344 ± 0.039 | 0.433 ± 0.039 | 0.465 ± 0.020 | 0.507 ± 0.016 | 0.340 ± 0.049 | 0.426 ± 0.044 | 0.552 ± 0.024 | 0.695 ± 0.013 |
| **STemDist** | **0.264 ± 0.006** | **0.337 ± 0.007** | **0.220 ± 0.005** | **0.292 ± 0.003** | **0.395 ± 0.003** | **0.443 ± 0.002** | **0.244 ± 0.006** | **0.314 ± 0.005** | **0.541 ± 0.011** | **0.675 ± 0.010** |
| Original data | 0.223 ± 0.003 | 0.282 ± 0.002 | 0.205 ± 0.002 | 0.282 ± 0.015 | 0.383 ± 0.007 | 0.425 ± 0.005 | 0.214 ± 0.003 | 0.283 ± 0.007 | 0.506 ± 0.007 | 0.661 ± 0.009 |

in Lemma 2. In further steps of the algorithm, computing the output for the synthetic dataset (line 13) takes $O(M_{\mathcal{S}}(\frac{N_{\mathcal{S}}^2}{K^2} + L_{in}(\frac{N_{\mathcal{S}}}{K})))$ time, and computing the output for the clustered dataset (line 11) takes $O(B(\frac{N_{\mathcal{S}}^2}{K^2} + L_{in}(\frac{N_{\mathcal{S}}}{K})))$ time by Lemma 1. Computing the loss for the synthetic dataset (line 14) takes $O(M_{\mathcal{S}}(\frac{N_{\mathcal{S}}}{K})L_{out}F_{out})$ time, and computing the loss for the original dataset (line 12) also takes $O(B(\frac{N_{\mathcal{S}}}{K})L_{out}F_{out})$ time, since we process $N_{\mathcal{S}}/K$ clusters per iteration rather than $N_{\mathcal{T}}$ individual locations. By Theorem 1, training the model takes $O(T_{model}M_{\mathcal{S}}(\frac{N_{\mathcal{S}}^2}{K} + N_{\mathcal{S}}L_{in}))$ time per distillation iteration. Putting them all together, the time cost of a single iteration for distillation is $O((T_{model}M_{\mathcal{S}} + B)(\frac{N_{\mathcal{S}}^2}{K}) + (L_{in}(T_{model}M_{\mathcal{S}} + B) + (M_{\mathcal{S}} + B)L_{out}F_{out})N_{\mathcal{S}})$. This process is conducted for $T_{distill}$ distillation iterations and $T_{outer}$ outer iterations, with clustering at the beginning of the process. Therefore, the total time complexity becomes $O(N_{\mathcal{T}}N_{\mathcal{S}}(M_{\mathcal{T}}L_{in}F)T_{kmeans} + T_{outer}T_{distill}((T_{model}M_{\mathcal{S}}+B)(\frac{N_{\mathcal{S}}^2}{K})+(L_{in}(T_{model}M_{\mathcal{S}}+B)+(M_{\mathcal{S}} + B)L_{out}F_{out})N_{\mathcal{S}}))$. □

### C. Training Details

In this section, we elaborate on the training details that were omitted in the main paper due to space constraints. We set the number of columns $R$ in the location embeddings to 10 for all methods. When using the location encoder model, the hidden dimension is set to 32. We set the batch size to 256 for GBA [7], 128 for GLA [7], 128 for ERA5 [8], 64 for CA [7], and 64 for CAMS [9] datasets. When evaluating the distilled dataset, we train MTGNN for 400 epochs for the GBA and GLA datasets, and 100 epochs for the other datasets. This also holds for evaluating the datasets distilled by the baselines.

### D. Hyperparameter Settings for Baselines

The hyperparameter settings of the baseline experiments that were not mentioned in our paper are mentioned in this section. We don't split the locations into subsets ($K$) when training the MTGNN [1] model on the synthetic dataset distilled by baselines, since this is the default option in the paper and the authors' code. When distilling with DC [4], we set the batch size to 128 for GBA, 64 for GLA, 64 for ERA5, 32 for CA, and 32 for CAMS datasets. Note that we use the maximum batch size that could be used while increasing the batch size by a factor of 2.

For MTT [10], we use the MTGNN model to generate training trajectories. A total of 20 expert buffers are created, and the trajectory in each buffer is generated by training for 15 epochs. While distilling with MTT, we set the "maximum start epoch" to 4, "number of expert epoch" to 2, and "number of synthetic steps" to 10, which was the most empirically effective combination in our experiments.

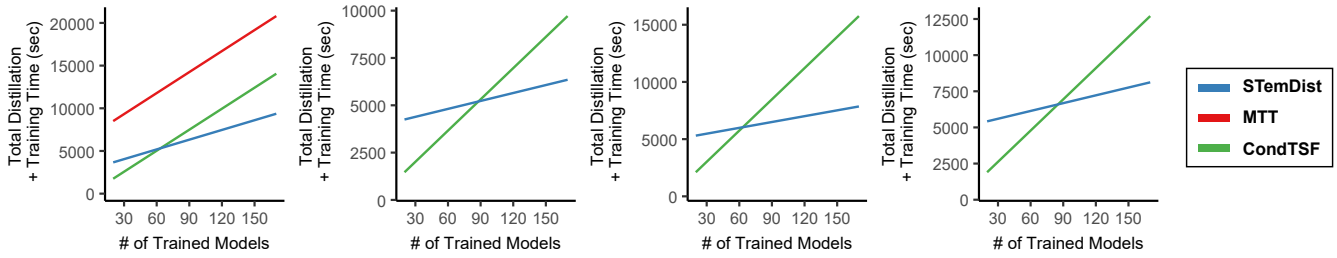
For CondTSF [5], DLinear [11] model was used to generate training trajectories. Similarly, 20 expert buffers were created, each trained for 15 epochs. The same hyperparameter configuration as in MTT was adopted (maximum start epoch = 4, number of expert epoch = 2, number of synthetic steps = 10) for the trajectory matching process. We set the additive update ratio to 0.01 and optimize the value term for every 3 epochs as the default setting in the paper.

For TimeDC [12], we used the model proposed in the TimeDC paper to generate training trajectories, with 20 expert buffers each trained for 40 epochs. All the other hyperparameters and settings for TimeDC experiments followed those specified in the paper.

The learning rates used to train the model on the synthetic data, to distill the synthetic data, and to train the model on the original data for creating expert trajectories are all selected via grid search over $\{10^{-2}, 10^{-3}, 10^{-4}\}$, except for TimeDC. For TimeDC experiments, we search within a larger range, from $10^{-2}$ to $10^{-7}$.

### E. Cross-model Performance across Additional Models

We report cross-model performance on additional spatio-temporal time series forecasting models, AGCRN [2] and




(a) GLA (b) ERA5 (c) CA (d) CAMS

Fig. 9. Total distillation and training cost with respect to the number of models trained using output synthetic datasets. Note that MTT runs out of memory on the ERA5, CA, and CAMS datasets.

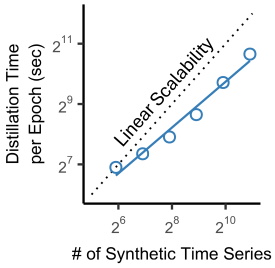
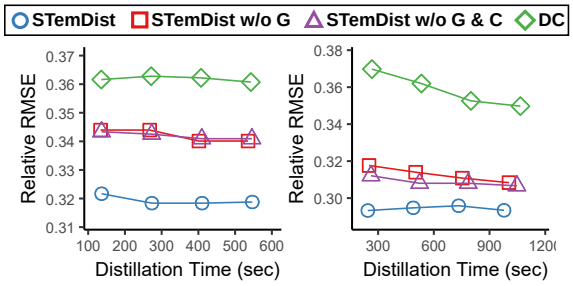



Fig. 10. Scalability of STemDist with respect to the number of synthetic time series. Its empirical distillation time increases sub-linearly with the number of synthetic time series.




(a) GBA (b) CA

Fig. 11. Ablation study. Distillation performance of STemDist and its variants over distillation time on the GBA and CA datasets.

STNORM [6] [1] in Table VI. The experimental setup is identical to that described in Section V-C of the main paper.

Note that out-of-memory errors occur when training AGCRN on the original dataset and on the synthetic datasets produced by all baselines, due to the large number of locations. AGCRN is successfully trained only on the synthetic dataset distilled by STemDist.

### *F. Scalability w.r.t. the Number of Synthetic Time Series*

In Figure 10, we report additional experimental results on the scalability of STemDist with respect to the number of synthetic time series. The experimental setup is identical to that described in Section V-D of the main paper.

### *G. Distillation and Training Cost*

In this section, we analyze the computational cost of dataset distillation methods, including the time required for dataset distillation and model training using output synthetic datasets.

*1) Settings:* For representative trajectory-matching-based methods, specifically MTT [10] and CondTSF [5], the *total distillation time* is measured as the sum of the time required to generate trajectories and the time required for distillation. MTT uses MTGNN [1] as the surrogate model in distillation, while CondTSF uses the model proposed in the paper, DLinear [11], as the surrogate model. Note that DLinear is a much lighter model than MTGNN. For MTT and CondTSF, we generate 20 expert trajectories, and 10 epochs are stored for each trajectory. For STemDist, the total distillation time is measured as the sum of the time required for spatial clustering and the time required for distillation. Across all methods and datasets, we perform distillation under a total compression ratio of 0.5% and run the distillation process for 100 epochs.

For all methods, we measure the *total training time*, defined as the total time spent on repeated model training using output synthetic datasets. Note that such repeated training is common in dataset distillation scenarios, where an output synthetic dataset is used to train multiple models under different hyperparameter configurations. We train each model for 400 epochs on the GBA and GLA datasets, and for 100 epochs on the ERA5, CA, and CAMS datasets.

*2) Results:* Figure 9 shows the total computational cost across of distillation methods, defined as the sum of distillation time and training time. The cost is reported for varying numbers of models trained using the output synthetic datasets. Note that MMT is evaluated only on the GLA dataset, as it runs out of memory on the others.

As shown in Figure 9(a), STemDist consistently shows shorter total distillation time compared to MTT, as MTT requires expensive trajectory learning processes. While CondTSF achieves a shorter distillation time due to its lightweight surrogate model, the higher training cost on its output synthetic dataset causes the total computation time to increase more rapidly as the number of trained models grows. STemDist produces synthetic datasets that enable more efficient model training, making it increasingly advantageous as more models are trained on the synthetic datasets. We observe that STemDist becomes more efficient than CondTSF when at least 64, 88, 64, and 86 models (or hyperparameter combinations) are trained on the GLA, ERA5, CA, and CAMS datasets, respectively, corresponding to the crossover points in Figure 9.

[1]STNORM is adjusted to normalize along the spatial dimension to handle time series with varying numbers of locations.

TABLE VII
DISTILLATION PERFORMANCE OF STEMDIST AND ITS VARIANT EMPLOYING AN MLP AS THE LOCATION ENCODER MODULE, REFERRED TO AS STEMDIST-MLP.

| Method | GBA | | GLA | | ERA5 | | CA | | CAMS | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Relative MAE | Relative RMSE | Relative MAE | Relative RMSE | Relative MAE | Relative RMSE | Relative MAE | Relative RMSE | Relative MAE | Relative RMSE |
| STemDist-MLP | 0.261 ± 0.003 | 0.323 ± 0.003 | 0.215 ± 0.003 | 0.280 ± 0.002 | 0.382 ± 0.003 | 0.432 ± 0.003 | 0.227 ± 0.010 | 0.290 ± 0.003 | 0.527 ± 0.013 | 0.654 ± 0.006 |
| STemDist | 0.251 ± 0.005 | 0.317 ± 0.005 | 0.211 ± 0.008 | 0.277 ± 0.002 | 0.385 ± 0.007 | 0.426 ± 0.004 | 0.226 ± 0.001 | 0.292 ± 0.002 | 0.522 ± 0.005 | 0.653 ± 0.005 |

TABLE VIII
EFFECT OF HYPERPARAMETERS ON THE DISTILLATION PERFORMANCE OF STEMDIST, MEASURED BY RELATIVE RMSE UNDER A TOTAL COMPRESSION RATIO OF 0.5%.

| Dataset | Temporal–Spatial Compression Pairs ($\frac{M_S}{M_T}$, $\frac{N_S}{N_T}$) | | | |
|---|---|---|---|---|
| | (2%, 25%) | (5%, 10%) | (10%, 5%) | (25%, 2%) |
| GBA | 0.318 ± 0.004 | 0.315 ± 0.004 | 0.317 ± 0.005 | 0.332 ± 0.003 |
| GLA | 0.279 ± 0.005 | 0.271 ± 0.003 | 0.277 ± 0.002 | 0.285 ± 0.005 |
| ERA5 | 0.426 ± 0.006 | 0.427 ± 0.008 | 0.426 ± 0.004 | 0.422 ± 0.004 |
| CA | 0.291 ± 0.003 | 0.296 ± 0.001 | 0.292 ± 0.002 | 0.301 ± 0.004 |
| CAMS | 0.655 ± 0.005 | 0.660 ± 0.008 | 0.653 ± 0.005 | 0.649 ± 0.004 |
| Avg. | 0.394 ± 0.005 | 0.394 ± 0.005 | 0.393 ± 0.004 | 0.398 ± 0.004 |

(a) Effect of temporal and spatial compression ratios

| Dataset | Number of Subsets $K$ | | | |
|---|---|---|---|---|
| | 2 | 4 | 8 | 16 |
| GBA | 0.328 ± 0.004 | 0.317 ± 0.005 | 0.323 ± 0.003 | 0.329 ± 0.003 |
| GLA | 0.279 ± 0.003 | 0.277 ± 0.002 | 0.280 ± 0.005 | 0.282 ± 0.005 |
| ERA5 | 0.426 ± 0.007 | 0.426 ± 0.004 | 0.423 ± 0.003 | 0.439 ± 0.008 |
| CA | 0.297 ± 0.006 | 0.292 ± 0.002 | 0.293 ± 0.003 | 0.291 ± 0.003 |
| CAMS | 0.662 ± 0.002 | 0.653 ± 0.005 | 0.656 ± 0.011 | 0.651 ± 0.005 |
| Avg. | 0.398 ± 0.004 | 0.393 ± 0.004 | 0.395 ± 0.005 | 0.398 ± 0.005 |

(b) Effect of the number of subsets

### H. Ablation Study on Additional Datasets

We report ablation results on additional datasets. The experimental setup is identical to that of the experiment in Section V.E of the main paper.

We compare STemDist with three variants:

- **STemDist w/o G**: STemDist without subset-based **G**ranular distillation.
- **STemDist w/o G & C**: STemDist without subset-based **G**ranular distillation and **C**lustering of locations.
- **DC** [4]: A naive gradient matching method.

Figure 11 shows how the performance of each variant changes over distillation time for GBA and CA datasets.

### I. Hyperparameter Analysis on Additional Settings

We examine the effect of hyperparameters on the distillation performance of STemDist under a fixed total compression ratio of 0.5% on the GBA, GLA, and ERA5 datasets. The experimental setup is identical to that described in Section V-F of the main paper, and the results are reported in Table VIII. Similar to the case with total compression ratio of 1% in Section V-F, these results also show that STemDist tends to be more effective when the temporal and spatial compression ratios are balanced, and achieves better performance with a moderate number of subsets (4 or 8).

### J. Distillation Performance for Rare Events

We analyze the test-set performance of MTGNN trained on the original dataset and on the synthetic dataset obtained by STemDist, stratified by the absolute values of the ground-truth targets. The experiments are conducted on the ERA5 dataset, where the forecasting target is wind speed. To categorize wind speed ranges, we adopt the Beaufort Wind Scale [13], a widely used classification in meteorology. The compression ratio of the distilled dataset is set to 0.5%.

TABLE IX
PREDICTION PERFORMANCE OF MODELS TRAINED ON THE REAL AND STEMDIST-GENERATED SYNTHETIC DATASETS ACROSS DIFFERENT GROUND-TRUTH WIND SPEED RANGES. THE ERROR METRIC USED IS MEAN SQUARED ERROR (MSE).

| Wind speed (Beaufort wind scale) | 0-3.3m/s (0-2) | 3.3-8m/s (3-4) | 8m/s- (5-) |
|---|---|---|---|
| STemDist | 0.949 | 1.253 | 4.073 |
| Original data | 0.992 | 1.317 | 3.918 |

As shown in Table IX, the two models achieve similar overall performance. The model trained on the synthetic dataset shows a slight relative performance drop in the extreme regime (where wind speeds exceed 8 m/s) and slight performance gains in the other regimes, but the differences are not substantial.

### K. Distillation Performance with a Simpler Location Encoder Design

To explore a simpler architecture for the location encoder module, we examine a variant of STemDist that uses an MLP as the location encoder. The MLP is applied independently to each location and is thus applicable to settings with varying numbers of locations. Specifically, it consists of two linear layers with a hidden dimension of 64 and is applied as follows:

$$\begin{aligned} E_{\mathcal{T}}(i,:) &= \mathrm{Linear}_1(\mathrm{Linear}_2(I_{\mathcal{T}}(i,:))). \\ E_{\mathcal{S}}(i,:) &= \mathrm{Linear}_1(\mathrm{Linear}_2(I_{\mathcal{S}}(i,:))), \end{aligned} \tag{1}$$

which can be used to replace Eq. (4) in the main paper. Recall that $I_{\mathcal{T}}$ and $I_{\mathcal{S}}$ are the inputs of location encoders. As shown in Table VII, while the variant referred to as STemDist-MLP underperforms STemDist, it achieves comparable overall performance. This result suggests that STemDist is relatively insensitive to the specific choice of location encoder design.

## REFERENCES


[1] Z. Wu, S. Pan, G. Long, J. Jiang, X. Chang, and C. Zhang, "Connecting the dots: Multivariate time series forecasting with graph neural networks," in *KDD*, 2020.

[2] L. Bai, L. Yao, C. Li, X. Wang, and C. Wang, "Adaptive graph convolutional recurrent network for traffic forecasting," in *NeurIPS*, 2020.

[3] R. Z. Farahani and M. Hekmatfar, *Facility location: concepts, models, algorithms and case studies*. Springer Science & Business Media, 2009.

[4] B. Zhao, K. R. Mopuri, and H. Bilen, "Dataset condensation with gradient matching," in *ICLR*, 2021.

[5] J. Ding, Z. Liu, G. Zheng, H. Jin, and L. Kong, "Condtsf: one-line plugin of dataset condensation for time series forecasting," *NeurIPS*, 2024.
[6] J. Deng, X. Chen, R. Jiang, X. Song, and I. W. Tsang, "St-norm: Spatial and temporal normalization for multi-variate time series forecasting," in *KDD*, 2021.
[7] Y. Fang, Y. Liang, B. Hui, Z. Shao, L. Deng, X. Liu, X. Jiang, and K. Zheng, "Efficient large-scale traffic forecasting with transformers: A spatial data management perspective," in *KDD*, 2025.
[8] H. Hersbach, B. Bell, P. Berrisford, G. Biavati, A. Horányi, J. Muñoz Sabater, J. Nicolas, C. Peubey, R. Radu, I. Rozum, D. Schepers, A. Simmons, C. Soci, D. Dee, and J.-N. Thépaut, "Era5 hourly data on single levels from 1940 to present," Copernicus Climate Change Service (C3S) Climate Data Store (CDS), 2023.
[9] A. Inness, M. Ades, A. Agustí-Panareda, J. Barré, A. Benedictow, A.-M. Blechschmidt, J. J. Dominguez, R. Engelen, H. Eskes, J. Flemming *et al.*, "The cams reanalysis of atmospheric composition," *Atmospheric Chemistry and Physics*, vol. 19, no. 6, 2019.
[10] G. Cazenavette, T. Wang, A. Torralba, A. A. Efros, and J.-Y. Zhu, "Dataset distillation by matching training trajectories," in *CVPR*, 2022.
[11] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?" in *AAAI*, 2023.
[12] H. Miao, Z. Liu, Y. Zhao, C. Guo, B. Yang, K. Zheng, and C. S. Jensen, "Less is more: Efficient time series dataset condensation via two-fold modal matching," *PVLDB*, vol. 18, no. 2, pp. 226–238, 2024.
[13] World Meteorological Organization, *Guide to Meteorological Instruments and Methods of Observation*, wmo-no. 8 ed. WMO, 2018.